NumPy-Only MICHD Classification: Feature Selection and Boosted-Tree Ensembles

Guodong MA, Seohyun LEE, Anthony YAGHI
guodong.ma@epfl.ch, seo.lee@epfl.ch, anthony.yaghi@epfl.ch

***Abstract*—We extend an original NumPy implementation of histogram Newton boosting with feature selection, categorical expansion and probability ensembles. The selected system achieves internal holdout F1 0.4445, compared with 0.4446 for the earlier model. Selection uses tuning predictions only; the previously inspected holdout is used for descriptive comparison. All training and inference use NumPy and standard Python. The code includes the six required course algorithms, reproducible model files and 109,379 test predictions.**

I. Data and evaluation

The official archive contains 328,135 labeled records, 109,379 test records and 321 input features after excluding the CSV ID. Positive MICHD labels account for 8.83% of records; 44.79% of feature entries are missing. Labels are aligned by ID and encoded as 0/1. Submission rows retain the official test order and use labels $-1/+1$.

We retain the original seed-2026 stratified 60/20/20 split: 196,881 training, 65,627 tuning and 65,627 holdout records. The holdout was inspected during earlier development, so it is not a pristine external test. All additional configurations are predefined; only tuning scores determine model, stopping rounds and threshold. The selected system is written to disk before new holdout predictions are computed. We do not change the selection in response to holdout results.

For each candidate, we maximize positive-class F1 over all distinct tuning-score cutoffs, grouping equal scores. We report precision, recall and accuracy as well. A high accuracy alone is uninformative here: always predicting negative gives accuracy 0.9117 but F1 zero.

II. Models and optimization

Original boosting. For logistic probability p_i, the gradient and Hessian are $g_i = p_i - y_i$ and $h_i = p_i(1-p_i)$. Each leaf outputs $-G/(H+\lambda)$, with summed gradient G, Hessian H and $\lambda = 10$. The split gain is

$$\frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right].$$

NumPy histogram accumulation evaluates numeric split thresholds and both missing-value directions. Quantile edges are fitted on training rows only. This is an independently written simplified algorithm, not the XGBoost library.

Feature views. The raw view uses all 321 features. The top-80 view sums split gains in the original training-fitted depth-three model and retains the 80 highest-ranked features. That reference was previously selected using tuning data; no holdout information enters feature ranking. The expanded view keeps these numeric features and adds one-hot columns for features with 2–12 distinct observed training values, plus the fraction of missing selected features. Unknown categories have zero one-hot values; missing values remain NaN. Category lists and selected columns are frozen before validation and reused at full refit.

Search. We compare the earlier 32-bin depth-three model with eight new configurations: 64 bins; 128 bins; 64 bins with depth four; 64 bins with column sampling 0.7; 64 bins with learning rate 0.05 and 450-round cap; top-80 with column sampling 0.8; expanded top-80 with sampling 0.5; and top-80 with depth four and sampling 0.8. Other settings are depth three, learning rate 0.1, 300-round cap, minimum child Hessian 20, row sampling 0.7, column sampling 0.35 and seed 2026. Every ten rounds, tuning F1 is checked; patience is 60 rounds with minimum improvement 10^{-6}.

We also test equal-weight probability averages of the best two and best three single models ranked by tuning F1. Each ensemble gets its own tuning-selected cutoff. The best tuning F1 among all single and ensemble candidates determines the final choice.

III. Results

Configuration	Tune F1	Holdout F1	Recall	Accuracy
Original reference	0.4483	0.4446	0.5534	0.8779
64 bins	0.4481	0.4408	0.5337	0.8804
128 bins	0.4485	0.4425	0.5377	0.8804
Depth 4	0.4487	0.4418	0.5418	0.8791
Column sampling 0.7	0.4479	0.4366	0.4935	0.8875
Learning rate 0.05	0.4487	0.4449	0.5596	0.8767
Top 80 features	0.4459	0.4437	0.5658	0.8747
Top 80 + one-hot	0.4474	0.4426	0.5627	0.8749
Top 80, depth 4	0.4454	0.4466	0.5603	0.8774
Best-two average	0.4494	0.4445	0.5584	0.8767
Best-three average	0.4488	0.4447	0.5558	0.8774

Table I
ALL TRIALS. BOLD MARKS THE CHOICE MADE ON TUNING DATA BEFORE HOLDOUT SCORING.

The selected Best-two average uses cutoff 0.207527, with holdout precision 0.3692 and F1 difference -0.0001 from the

reference. After selection, the team reported AIcrowd submission #338129: F1 0.452 and accuracy 0.878 (displayed precision). This online result is recorded separately; it uses a different evaluation set and does not establish an online gain over the reference.

Uncertainty. A paired bootstrap with 2,000 replicates (seed 2026) resamples the joint categories of true label, reference prediction and selected prediction. The 95% percentile interval for the F1 difference is [-0.0034, +0.0034]. This conditional interval excludes training and selection variability and does not correct for holdout reuse. A small observed difference should not be interpreted as robust superiority.

Tradeoffs. The tuning improvement did not transfer to the holdout. Neither more bins nor one-hot expansion improved on the reference there. Top-80 depth-four trees had the highest holdout score, but we retain the tuning-selected ensemble rather than select retrospectively on holdout. All trials, including unsuccessful variants, are reported.

IV. Reproducibility and limitations

The selected members use 200 depth-four trees and 350 depth-three trees at learning rates 0.1 and 0.05. Both are independently refitted on all labeled rows. Feature views stay fixed; quantile cuts are refitted. Their equal-weight mean is thresholded once. The submitted file has 109,379 rows and 14,771 positive predictions.

The repository retains the two final models, their training configuration, and recorded comparison metrics. Earlier exploratory code remains in Git history. Inference checks model integrity and feature order; the training entry refits the selected members using the saved configuration and cutoff. We use NumPy 2.3.5 and Python 3.12, without external ML libraries, datasets or pretrained external models.

The teammate's `implementations.py` supplies the six mandatory algorithms, with an original snapshot preserved. Changes are formatting and singular-matrix fallback. `base_models.py` provides shared fit/predict entry points. Required signatures, batch-size-one SGD and unpenalized returned losses are retained.

This experiment classifies reported disease history. Numeric survey codes, survey-design features and missing patterns can act as proxies; questionnaire-specific nonresponse codes were not fully decoded. We have not established geographic, temporal or clinical generalization. Repeated tuning on one tuning split can overfit model selection. Further evidence would require nested or repeated validation, rather than repeated submissions to the public leaderboard.

V. Submission status and sources

Code and report were developed with AI assistance. Code is available in the public GitHub repository; course-system submission is pending. Runtime dependencies satisfy the stated NumPy-only restriction.

[1] EPFL Project 1 materials and supplied Fall 2026 description.
[2] Boosted-tree mathematical objective. Principles only; no tree implementation copied.
[3] AIcrowd challenge. Accessed 28 October 2026.